

Program1:

```
import pandas as pd
file_path_csv = r"C:\Users\student\Desktop\DSBAS\msc.csv"
file_path_excel = r"C:\Users\student\Desktop\DSBAS\FSI-2023-DOWNLOAD.xlsx"
file_path_text = r"C:\Users\student\Desktop\DSBAS\msc.txt"
df_csv = pd.read_csv(file_path_csv)
print("CSV File:")
print(df_csv)
df = pd.read_excel(file_path_excel)
print("Excel File:")
print(df.head())
with open(file_path_text, "r") as f:
    text = f.read()
print("\n Text file:")
print(text)
```

Data sets:

msc.csv:

register,name,mark

1,arun,89

2,jefresen,87

3,sethu,89

4,gokul,97

5,kumar,86

msc.xlsx:

Register	Name	Mark
----------	------	------

1	Arun	89
---	------	----

2	Kumar	86
---	-------	----

3	Ram	97
---	-----	----

4	Gokul	78
---	-------	----

5	Hari	77
---	------	----

msc.txt:

Register	Name	Mark
----------	------	------

1	Arun	89
---	------	----

2	Kumar	86
---	-------	----

3	Ram	97
---	-----	----

4	Gokul	78
---	-------	----

5	Hari	77
---	------	----

Output:

CSV File:

	register	name	mark
--	----------	------	------

0	1	arun	89
---	---	------	----

1	2	jefresen	87
---	---	----------	----

2	3	sethu	89
---	---	-------	----

3	4	gokul	97
---	---	-------	----

4	5	kumar	86
---	---	-------	----

Excel File:

	Register	Name	Mark
--	----------	------	------

0	1	Arun	89
---	---	------	----

1	2	Kumar	86
---	---	-------	----

2	3	Ram	97
---	---	-----	----

3	4	Gokul	78
---	---	-------	----

4	5	Hari	77
---	---	------	----

Text file:

Register	Name	Mark
1	Arun	89
2	Kumar	86
3	Ram	97
4	Gokul	78
5	Hari	77

Program 2:

```
import pandas as pd
import numpy as np
data = {
'A': [1, 2, 3, 4, 5],
'B': [10, 20, 30, 40, 50],
'C': [2.5, 3.5, 1.5, 4.5, 2.0]
}
df = pd.DataFrame(data)
mean_values = df.mean()
print("Mean values:")
print(mean_values)
print()
mode_values = df.mode()
print("Mode values:")
print(mode_values)
print()
median_values = df.median()
print("Median values:")
print(median_values)
```

```
print()
std_values = df.std()
print("Standard deviation values:")
print(std_values)
print()
summary_stats = df.describe()
print("Summary statistics:")
print(summary_stats)
```

output:

Mean values:

```
A    3.0
B   30.0
C    2.8
```

dtype: float64

Mode values:

```
   A  B  C
0  1  10  1.5
1  2  20  2.0
2  3  30  2.5
3  4  40  3.5
4  5  50  4.5
```

Median values:

```
A    3.0
```

B 30.0

C 2.5

dtype: float64

Standard deviation values:

A 1.581139

B 15.811388

C 1.204159

dtype: float64

Summary statistics:

	A	B	C
count	5.000000	5.000000	5.000000
mean	3.000000	30.000000	2.800000
std	1.581139	15.811388	1.204159
min	1.000000	10.000000	1.500000
25%	2.000000	20.000000	2.000000
50%	3.000000	30.000000	2.500000
75%	4.000000	40.000000	3.500000
max	5.000000	50.000000	4.500000

program3:

```
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
x = np.linspace(0, 10, 100)
y1 = np.sin(x)
y2 = np.cos(x)
plt.figure(figsize=(14, 10))
plt.subplot(3, 2, 1)
plt.plot(x, y1, label='sin(x)', color='b')
plt.plot(x, y2, label='cos(x)', color='r', linestyle='--')
plt.title('Line Plot')
plt.legend()
plt.subplot(3, 2, 2)
plt.scatter(x, y1, color='g', label='sin(x)')
plt.scatter(x, y2, color='m', label='cos(x)')
plt.title('Scatter Plot')
plt.legend()
categories = ['A', 'B', 'C', 'D']
values = [25, 30, 35, 40]
plt.subplot(3, 2, 3)
plt.bar(categories, values, color='pink')
plt.title('Bar Plot')
data = np.random.normal(0, 1, 1000)
plt.subplot(3, 2, 4)
plt.hist(data, bins=30, color='orange', edgecolor='black')
plt.title('Histogram')
sizes = [15, 30, 45, 10]
labels = ['Apple', 'Orange', 'Banana', 'Mango']
plt.subplot(3, 2, 5)
```

```
plt.pie(sizes, labels=labels, autopct='%1.1f%%', colors=['gold', 'yellowgreen',  
'lightcoral', 'lightskyblue'])
```

```
plt.title('Pie Chart')
```

```
plt.subplot(3, 2, 6)
```

```
data1 = np.random.normal(0, 1, 100)
```

```
data2 = np.random.normal(0, 2, 100)
```

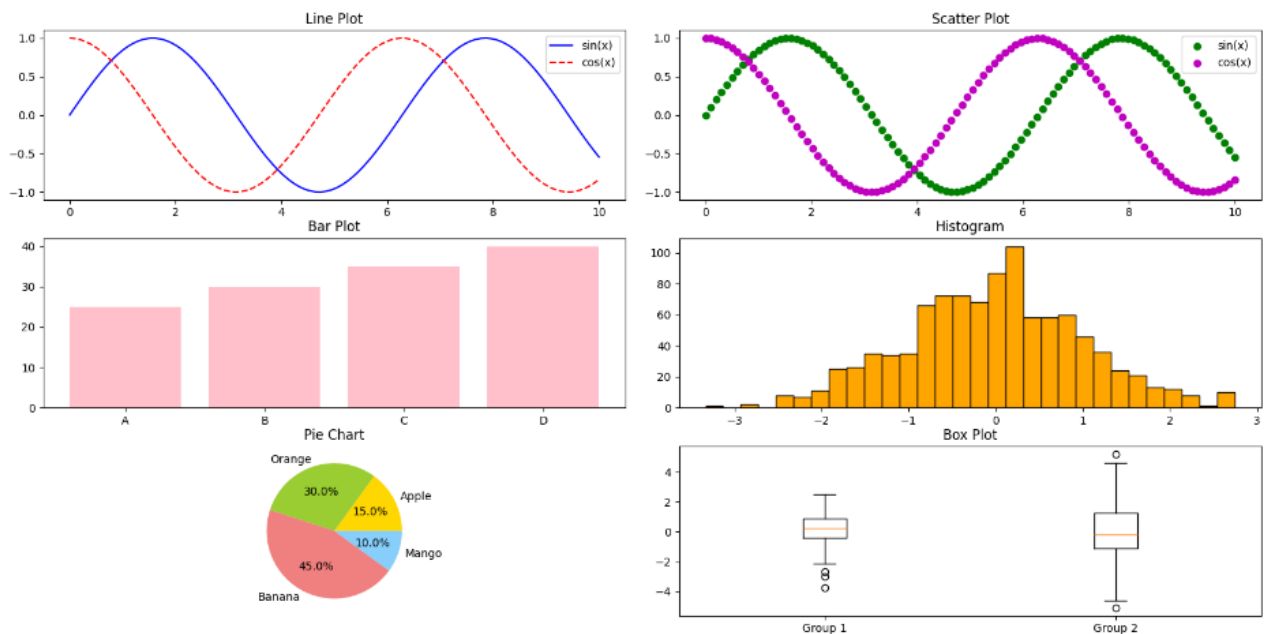
```
plt.boxplot([data1, data2], labels=['Group 1', 'Group 2'])
```

```
plt.title('Box Plot')
```

```
plt.tight_layout()
```

```
plt.show()
```

output:



program4:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
df = pd.read_csv(r'C:\Users\student\Desktop\DSBAS\pgm2.csv')
p= df[['s.no', 'marks']].values
kmeans = KMeans(n_clusters=4)
kmeans.fit(p)
y_kmeans = kmeans.predict(p)
centers = kmeans.cluster_centers_
plt.figure(figsize=(8, 6))
plt.scatter(p[:, 0], p[:, 1], c=y_kmeans, s=50)
plt.scatter(centers[:, 0], centers[:, 1], c='red', s=100, alpha=0.75, marker='X')
plt.title('K-means clustering')
plt.xlabel('s.no')
plt.ylabel('marks')
plt.show()
```

Data sets:

Pgm2.csv:

s.no,marks

1,87

2,89

3,67

4,76

5,87

6,90

7,87

8,34

9,97

10,56

12,57

13,58

14,89

15,67

16,78

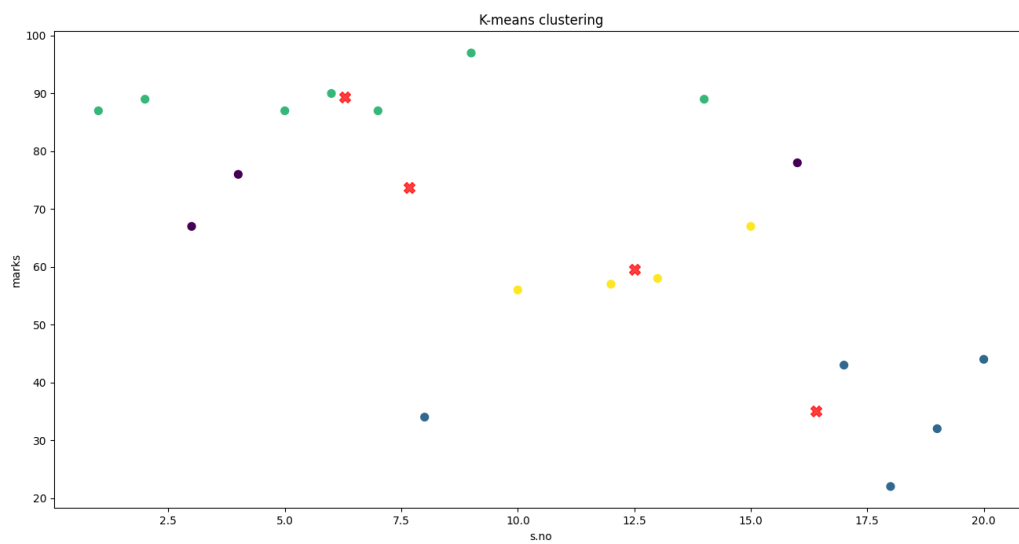
17,43

18,22

19,32

20,44

Output:



program5:

```
import matplotlib.pyplot as plt

from sklearn.neighbors import KNeighborsClassifier

x = [4, 5, 10, 4, 3, 11, 14, 8, 10, 12]
y = [21, 19, 24, 17, 16, 25, 24, 22, 21, 21]
classes = [0, 1, 1, 0, 0, 1, 1, 0, 1, 1]
data = list(zip(x, y))

print(data)

knn = KNeighborsClassifier(n_neighbors= 1)
knn.fit(data, classes)

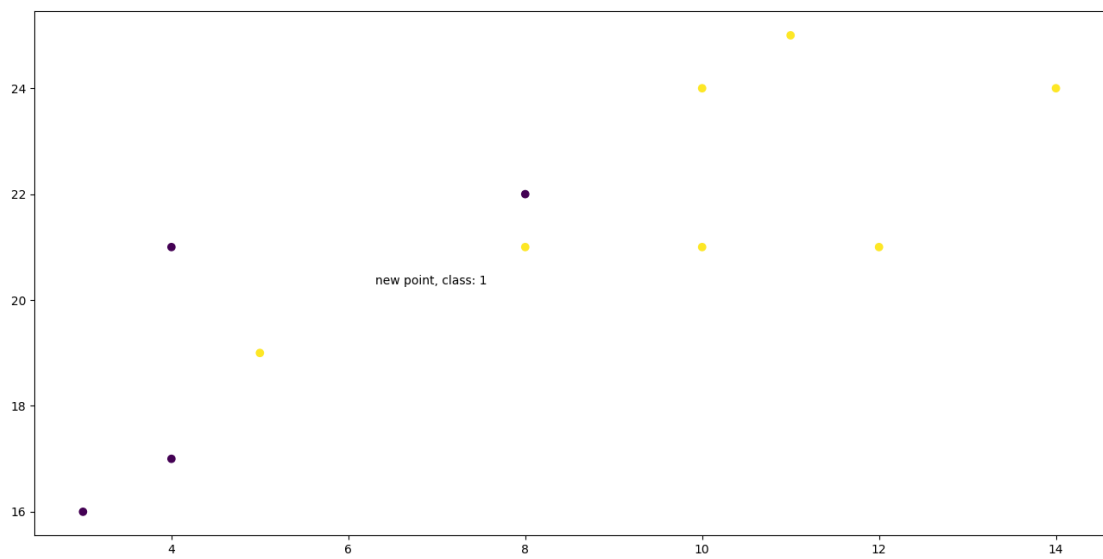
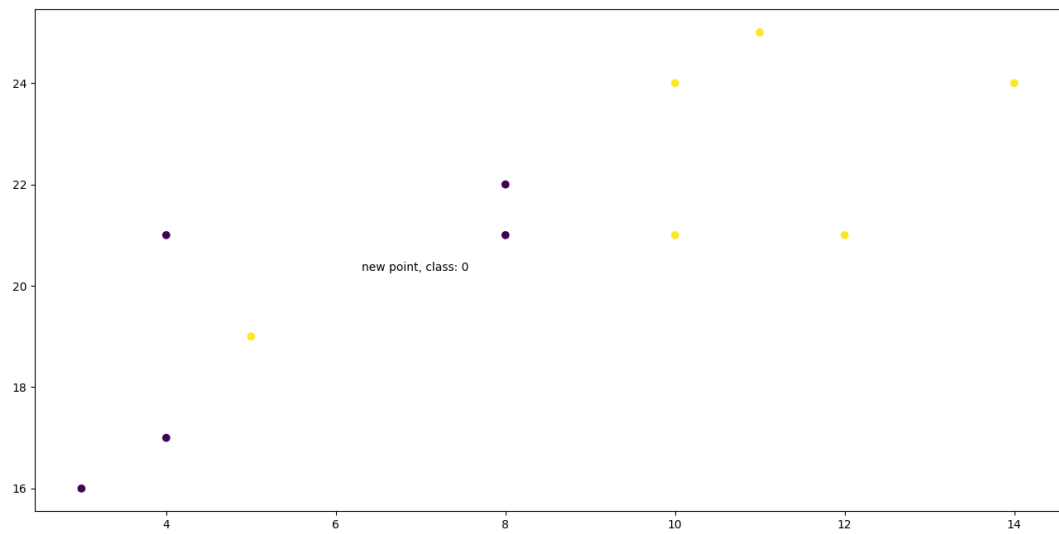
new_x = 8
new_y = 21
new_point = [(new_x, new_y)]
prediction = knn.predict(new_point)
print(prediction)

plt.scatter(x + [new_x], y + [new_y], c=classes + [prediction[0]])
plt.text(x=new_x-1.7, y=new_y-0.7, s=f"new point, class: {prediction[0]}")
plt.show()

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(data, classes)
prediction = knn.predict(new_point)
print(prediction)

plt.scatter(x + [new_x], y + [new_y], c=classes + [prediction[0]])
plt.text(x=new_x-1.7, y=new_y-0.7, s=f"new point, class: {prediction[0]}")
plt.show()
```

output:



program6:

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler, LabelEncoder

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

file_path = r'C:\Users\student\Desktop\DSBAS\dataset.csv'

df = pd.read_csv(file_path)

print(df.head())

encoder = LabelEncoder()

df['Name'] = encoder.fit_transform(df['Name'])

X = df[['Registerno', 'Name', 'Marks']]

y = df['Result']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=0)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)

model = LogisticRegression()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print(f"Accuracy: {accuracy * 100:.2f}%")

print("\nClassification Report:")

print(classification_report(y_test, y_pred))

print("\nConfusion Matrix:")

print(confusion_matrix(y_test, y_pred))
```

Data sets:

dataset.csv:

Registerno,Name,Marks,Result

1,Arun,350,pass

2,Charan,400,pass

3,Gokul,360,pass

4,Hari,370,pass

5,vel,440,pass

6,Santhose,450,fail

7,Sunil,300,fail

8,Prakash,270,fail

9,Somehwaran,350,fail

Output:

Registerno Name Marks Result

0 1 Arun 350 pass

1 2 Charan 400 pass

2 3 Gokul 360 pass

3 4 Hari 370 pass

4 5 vel 440 pass

Accuracy: 100.00%

Classification Report:

precision recall f1-score support

fail 1.00 1.00 1.00 1

pass 1.00 1.00 1.00 2

accuracy 1.00 3

macro avg	1.00	1.00	1.00	3
weighted avg	1.00	1.00	1.00	3

Confusion Matrix:

[[1 0]

[0 2]]

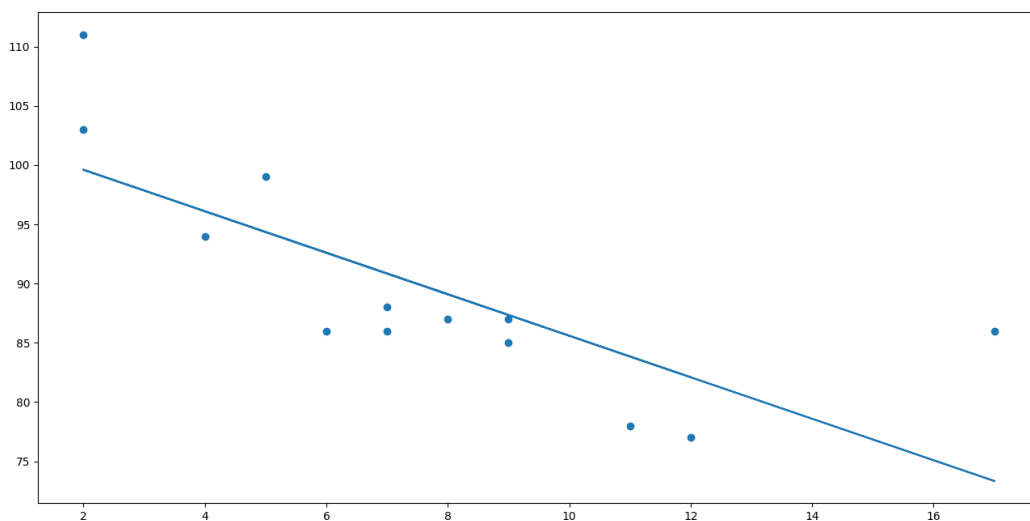
Program7:

```
import matplotlib.pyplot as plt
from scipy import stats
x=[5,7,8,7,2,17,2,9,4,11,12,9,6]
y=[99,86,87,88,111,86,103,87,94,78,77,85,86]
slop,intercept,r,p,std_err=stats.linregress(x,y)
print(r)
def myfunc(x):
    return slop*x+intercept
mymodel=list(map(myfunc,x))
speed=myfunc(10)
print(speed)
plt.scatter(x,y)
plt.plot(x,mymodel)
plt.show()
```

output:

-0.758591524376155

85.59308314937454



program8:

```
from sklearn import preprocessing
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt

data = [
    ['sunny', 85, 'high', False, 'no'],
    ['sunny', 80, 'normal', True, 'no'],
    ['overcast', 83, 'high', False, 'yes'],
    ['rainy', 70, 'high', False, 'yes'],
    ['rainy', 68, 'normal', False, 'yes'],
    ['rainy', 65, 'normal', True, 'no'],
    ['overcast', 64, 'high', True, 'yes'],
    ['sunny', 72, 'high', False, 'no'],
    ['sunny', 69, 'high', True, 'yes'],
    ['rainy', 75, 'high', True, 'no'],
    ['sunny', 75, 'normal', False, 'yes'],
    ['overcast', 72, 'normal', True, 'yes'],
    ['overcast', 81, 'high', False, 'yes'],
    ['rainy', 71, 'normal', True, 'no']
]

le = preprocessing.LabelEncoder()
X = []

for entry in data:
    X.append([le.fit_transform([entry[0]])[0], entry[1], le.fit_transform([entry[2]])[0],
    entry[3]])

y = [entry[4] for entry in data]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    random_state=42)

clf = DecisionTreeClassifier(random_state=42)
clf.fit(X_train, y_train)
```



```

plt.figure(figsize=(10, 6))

plot_tree(clf, filled=True, feature_names=['Outlook', 'Temperature', 'Humidity',
'Windy'])

plt.title("Decision Tree Visualization - Golf Dataset")

plt.show()

y_pred = clf.predict(X_test)

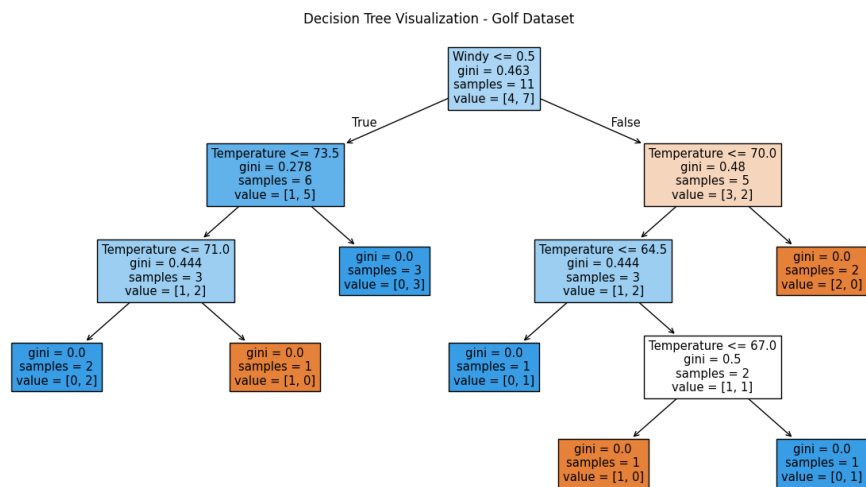
accuracy = sum(y_pred == y_test) / len(y_test)

print(f"Accuracy on test set: {accuracy:.2f}")

```

output:

Accuracy on test set: 0.33



program9:

```
from sklearn.naive_bayes import CategoricalNB
import numpy as np
X = np.array([
    [0, 0, 0, 0],
    [0, 0, 0, 1],
    [1, 0, 0, 0],
    [2, 1, 0, 0],
    [2, 2, 1, 0],
    [2, 2, 1, 1],
    [1, 2, 1, 1],
    [0, 1, 0, 0],
    [0, 2, 1, 0],
    [2, 1, 1, 0],
    [0, 1, 1, 1],
    [1, 1, 0, 1],
    [1, 0, 1, 0],
    [2, 1, 0, 1]
])
y = np.array([0, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1, 0])
clf = CategoricalNB()
clf.fit(X, y)
X_test = np.array([[0, 2, 0, 0]])
predicted = clf.predict(X_test)
if predicted[0] == 1:
    print("Prediction: Play Golf")
else:
    print("Prediction: Don't Play Golf")
```

output:

Prediction: Don't Play Golf

Prediction: Play Golf

program10:

```
def text_analysis(paragraph):
    total_chars = len(paragraph)
    words = paragraph.split()
    total_words = len(words)
    sentences = paragraph.split('.')
    total_sentences = len(sentences)
    if total_words > 0:
        avg_word_length = total_chars / total_words
    else:
        avg_word_length = 0
    and_count = paragraph.lower().count("is")
    print("Text Analysis Report")
    print("=====")
    print(f"Total characters (including spaces and punctuation): {total_chars}")
    print(f"Total words: {total_words}")
    print(f"Total sentences: {total_sentences}")
    print(f"Average word length: {avg_word_length:.2f}")
    print(f"Occurrences of 'is': {and_count}")
if __name__ == "__main__":
    paragraph = ""
    Education is the cornerstone of personal growth and societal progress.
    It empowers individuals with knowledge and skills essential for navigating
    the complexities of life. Beyond academics, education fosters critical thinking,
    creativity, and empathy, shaping individuals into informed and responsible global
    citizens. Accessible and quality education is pivotal in bridging socio-economic
```

gaps and building a brighter, more inclusive future for generations to come.

"""

text_analysis(paragraph)

output:

Text Analysis Report

=====

Total characters (including spaces and punctuation): 501

Total words: 62

Total sentences: 5

Average word length: 8.08

Occurrences of 'is': 2